Assignment: build a small agent service the way a platform team would

Build an assistant that helps employees with IT support tickets. It answers questions about tickets and can act on them once the user has approved the action. Package it so it could run on Amazon Bedrock AgentCore Runtime without code changes.

You don't need an AWS account, a paid LLM subscription or any company setup. Everything runs on your laptop, and that is how we will run it too, with our own model credentials.

This is a platform role. Other teams will copy what you build, so build it the way you would want them to copy it.

What to build

1. **The agent service.** An HTTP service in a container that follows the AgentCore Runtime contract in `CONTRACT.md`: `POST /invocations` and `GET /ping` on port 8080. The session id comes in a header; keep memory per session so a conversation holds together across turns. Use any agent framework or none, and any LLM provider.
2. **The tool layer.** One MCP server over the mock ticketing API in the kit: search tickets, get a ticket, create a ticket, update a ticket, add a comment. You choose the tool names, schemas and descriptions, and the agent should pick the right tool because of them. The API needs an API key, paginates, and has a `FLAKY=1` mode that returns 429s and 500s. Handle that the way you would in production.
3. **An approval step.** Any write (create a ticket, add a comment, change a status) needs the user's confirmation first. The agent proposes, the user confirms in the next turn, the agent executes. It should cope when the user declines or changes their mind. *Good to have:* it also copes when the user asks something unrelated before answering.
4. **An evaluation set.** A handful of cases you can run with one command: the right tool gets chosen, the approval step holds, out-of-scope requests are declined, and at least one case runs with `FLAKY=1`. Start from `eval/cases.sample.json` and put the results in your README. *Good to have:* a FLAKY case that shows a retried create does not produce duplicate tickets.
5. **Logs.** Structured logs for every invocation: the request id, the session id, and each tool call with its outcome and latency. *Good to have:* token usage, and tracing with OpenTelemetry or similar.
6. **A web UI.** A chat page against your service that shows the tool calls for each turn (name, arguments, outcome) and lets the user accept or decline an approval. We want to see what the agent is doing. We are not judging the visual design.

If you want to go further, pick one: memory across sessions (say what is stored, for how long, and who can see it); a second tool source; streaming responses; or an infrastructure-as-code skeleton for the AWS deployment that synthesises or validates without an account.

Starter kit

The kit gives everyone the same target:

- `mock-ticketing/`: the ticketing system as a Docker Compose service on port 8000. API key, pagination, 24 seeded tickets, the `FLAKY=1` mode, a reset endpoint for your evals, and the OpenAPI spec.
- `CONTRACT.md`: the Runtime contract and the request/response shape we will use when we run your service.
- `eval/cases.sample.json`: sample cases to start from. The tool names in it are placeholders; use yours.

Copy the kit into your repo so a single `docker compose up` starts the ticketing API and your service together. Don't change the ticketing API itself. If you have to, tell us what you changed and why.

Choices you're free to make

- **Language:** Python or TypeScript, whichever you are stronger in. Both have MCP SDKs and agent frameworks.
- **Framework:** Strands Agents, LangGraph, CrewAI, or the provider SDK on its own. Tell us why.
- **LLM provider:** any you can get at, including free tiers and local models through Ollama. Read the provider and model from environment variables and include a `.env.example`; we will run with our own credentials.
- **UI:** anything. A single page served by your service is fine.

What to submit

1. GitHub repo with your code.
2. README.md / other docs in the repo with:
    a. Quick setup instructions: how to start everything (ideally one `docker compose up`) and how to run the evals.
    b. Architecture overview (a simple diagram is great but not required).
    c. How this would run on AgentCore: Runtime, Memory, Gateway, Identity, Observability. What changes, what doesn't, and what you would need from the platform team.
    d. Agent and LLM decisions: framework, model, prompt and context management, tool schema and description design, guardrails, how you measured quality.
    e. Key technical decisions you made and why.
    f. Engineering standards you've followed (and maybe some that you skipped).
    g. How you used AI tools in your development process.
    h. What you'd do differently with more time.
    i. *Note: we need your thoughts, not an LLM's direct output.*

3. Screenshots of your UI, including an approval turn. If time permits, a short video.
4. Your eval results.

What we're looking for

- **The contract holds.** `docker build`, `docker run -p 8080:8080`, curl `/ping`, curl `/invocations`, and it works.
- **The tool layer.** Names, schemas and descriptions that make the right tool obvious. Auth, pagination, errors, retries and rate limits handled on purpose.
- **State and failure handling.** The approval step survives a real conversation, and the agent behaves when the API misbehaves.
- **Evaluation.** You measured it and can show what passed, what failed and what you changed.
- **Logs** we can follow from one request to the end.
- **A UI** that shows what the agent is doing, including the approval.
- **Engineering.** Clean, readable, tested, containerised code another team could copy.
- **AI-assisted development.** How you used AI coding tools, how you kept the output to your standard, and your do's and don'ts.
- **Your thinking.** The write-up reads like you, and the AgentCore section shows you know where this would live.

Tips & Notes

- **Start simple, then enhance.** A basic working version with great engineering beats a complex broken one. We won't judge you on every edge case; note the ones you skipped in your README.
- **Don't deploy anything.** We run your submission locally.
- **Out of time?** Document what you'd add next.
- **In the technical interview** you will walk us through your submission and we will ask you to change something live. Build something you can explain.
- **We want your thoughts, not an LLM's output.** Use AI assistants for the code as much as you like, and decide for yourself where the write-up has to be in your own words.

We value a solid, well-engineered basic solution a lot more than an over-engineered complex one.